

Modelantwoorden op juryvragen

Geautomatiseerde security assessment van MCP-servers

Verdediging bachelorproef — Arthur Van Oudenhove

Afgestemd op de **ingediende** (FINAL) versie: één gecontroleerde evaluatiecase (Goat), met de Zowe-scan als werk *na* indiening om de externe validiteit te toetsen. De drie zwaarste vragen ken je vloeiend van lijn (niet woord-voor-woord). De rest moet je *kunnen*, niet uit het hoofd kennen.

Twee grondregels. (1) Nooit live data verzinnen. (2) Nooit een fout verdedigen waarvan je weet dat het er een is: erken hem, corrigeer hem mondeling, toon dat je de juiste versie kent. Een erkende-en-gecorrigeerde fout scoort beter dan een fout die er nooit was.

Deel 1. De drie zwaarste vragen

Deze gaan met grote zekerheid komen en wegen het zwaarst op je cijfer. De lijn is telkens: *erkennen, ontwapenen, terugvallen op wat robuust is*.

1. “Waar is je tweede case?” / externe validiteit

In de ingediende versie is de “twee cases”-belofte geschraapt, dus de tegenstrijdigheid hoef je niet meer te bekennen. De vraag komt hooguit als vraag naar externe validiteit, en daar schittert je Zowe-werk.

“Ik heb mijn evaluatie bewust afgebakend tot één gecontroleerde case met een vaste ground truth, de Goat-server. Dat staat zo in mijn onderzoeksvraag. De prijs daarvan is beperkte externe validiteit, en dat benoem ik expliciet. Net om die grens te toetsen heb ik na indiening de scanner op een echte server gedraaid, Zowe, en daar zag ik de precisie instorten van 92,3% naar bijna nul. Dat bevestigt empirisch wat ik in mijn beperkingen schrijf.”

2. “Je benchmark is circulair. Wat bewijst 100% recall dan?”

Dé methodologische zwakte, en je benoemt hem zelf in je beperkingen. Erken hem voller dan de jury en overtref hem met echte data.

“Mijn 92,3% precisie op Goat is een best-case: de benchmark en mijn detectieregels zijn samen ontworpen. Op een echte server, Zowe, zakt diezelfde precisie naar bijna nul. Een tier-sweep toont waarom: de autorisatie-bevindingen schalen mee met Zowe’s capability-tier-policy, niet met echte kwetsbaarheden, omdat mijn heuristieken autorisatie in het schema zoeken terwijl Zowe die op transportniveau afdwingt. De eerlijke conclusie: het framework is een snelle, reproduceerbare screening met hoge recall maar lage precisie buiten zijn benchmark. Het in kaart brengen van die grens beschouw ik als een even waardevol resultaat als de detectie zelf.”

3. “Je handmatige vergelijking is $n=1$, door jezelf. Vertekend?”

Toegeven, en terugvallen op de twee claims die niet van de reviewer afhangen: het tijdsverschil en de reproduceerbaarheid.

“Ja, en daarom presenteer ik die vergelijking expliciet als illustratief, niet als statistisch valide. Één reviewer, en bovendien de bouwer van de tool, wat naar de tool toe vertekent. Daarom leun ik niet op het recall-verschil van 5 op 5 tegenover 4 op 5. De claim die wél robuust is tegen reviewer-variatie is het tijdsverschil: ongeveer 30 seconden tegenover 47 minuten, een structureel verschil van factor negentig dat geen enkele plausibele reviewer wegneemt. En de reproduceerbaarheid is per constructie reviewer-onafhankelijk: dezelfde omgeving levert altijd identieke bevindingen, een menselijke review niet.”

Deel 2. Volledige vragencatalogus

Per vraag: waarom ze het stellen en de ruggengraat van je antwoord. Geen script, wel de lijn die je moet kunnen verdedigen.

A. Validiteit (hoogste inzet)

1. Circulaire benchmark — zie Deel 1.2.

2. n=1 reviewer, partijdig — zie Deel 1.3.

3. “Wat maakt dit Fintech-specifiek? Je regels zijn generiek MCP-security.” *Waarom:* Je titel is Fintech, maar SAST-001 t/m FUZZ-004 zijn niet financieel; alleen de Goat-scénario's (overschrijving, saldo) zijn dat.

Lijn: Eerlijk zijn: de detectiemechanismen zijn MCP-generiek; de Fintech-laag zit in (a) de requirementsanalyse die compliance-kaders (PCI-DSS, GDPR) naar criteria vertaalt, (b) de keuze van kwetsbaarheidsklassen die in financiële context kritiek zijn (ongeautoriseerde transfers, negatieve bedragen, V3/V4), en (c) de severity-weging. Niet pretenderen dat de regex Fintech “kent”.

B. Technische diepte (Ownership)

4. “SAST-001 zoekt met regex naar 'ignore rules'. Ik herformuleer mijn prompt injection en je tool ziet niks. Hoe robuust is dit?” *Waarom:* Pattern matching is triviaal te omzeilen; je geeft toe “detecteert patronen, geen intentie”.

Lijn: Klopt, en bewust. Regex-detectie heeft inherent recall-verlies bij geobfusceerde payloads, een bekende grens van SAST (Chess & McGraw). De keuze is bewust: deterministische, uitlegbare, snelle regels zonder LLM-afhankelijkheid, geschikt voor CI/CD. Een semantische/LLM-detector zou recall verhogen maar reproduceerbaarheid en snelheid opofferen. Een expliciete trade-off, geen blinde vlek.

5. “Je rapporteert 13 bevindingen, maar regels overlappen. Blaas je het aantal niet op?” *Waarom:* Één ground-truth-item wordt door meerdere regels geraakt; recall telt instances, niet unieke root causes.

Lijn: Onderscheid recall (dekking ground truth, 5/5 uniek) van bevindingen (13, met overlap). De overlap is geen inflatie maar defense-in-depth: dezelfde fout via statisch én dynamisch pad vangen verhoogt betrouwbaarheid. Je benoemt de overlap zelf transparant.

6. “Veel MCP-servers draaien over stdio. Werkt je scanner daar?” *Waarom:* Realistische dekkingsvraag; je architectuur gaat uit van HTTP.

Lijn: Eerlijk: de scanner ondersteunt HTTP/SSE-transport, niet stdio. Geef dat toe als scope-beperving en vervolgstap. Niet bluffen.

7. “Wat heb je zelf gebouwd, en wat komt uit libraries?” *Waarom:* Ownership-criterium, letterlijk in de rubric.

Lijn: Officiële `mcp-library` = transport/RPC; Pydantic = validatie. **Jouw werk** = de drie-module-architectuur, het uniforme `Finding`-model, de 8 detectieregels (4 SAST + 4 fuzzing), de baseline-argumentgeneratie uit JSON-schema, de rapportgenerator, de Goat-server zelf en de Docker-opstelling. Oefen dit als een vloeiende opsomming van 30 seconden.

C. Methodologische keuzes

8. “Burp en Semgrep schieten tekort: empirisch getest of uit literatuur?” *Waarom:* Deelvraag 2 maakt deze claim; als je het niet zelf testte, is het een aanname.

Lijn: Wees precies. Burp gebruikte je voor manuele verificatie, niet als vergelijkingsbaseline. Semgrep niet zelf op Goat gedraaid: presenteer de claim als literatuur-onderbouwd (deze tools kennen geen JSON-RPC-tool-semantiek), niet als eigen meetresultaat. Niet overclaimen.

9. “Je spreekt over de OWASP MCP Top 10. Bestaat die, en klopt je mapping?”

Waarom: Inconsistentie tussen “OWASP MCP Top 10” en de gebruikte LLM-codes. Dit is je enige resterende tekstuele exposure (zie Deel 3).

Lijn: Niet zelf aansnijden. Als het komt: erken dat de traceability-tabel de oudere OWASP LLM-Top-10-codes gebruikt terwijl je naar de MCP Top 10 verwijst, en geef meteen de correcte mapping (Deel 3). Dat je de juiste codes uit het hoofd kent, bewijst beheersing van de standaard.

10. “Waarom een eigen Goat-server in plaats van MCPTox (45 reële servers)?”

Waarom: Je noemt MCPTox als vervolgstap, dus de tegenvraag waarom niet meteen.

Lijn: Goat geeft een bekende, gecontroleerde ground truth die je voor recall-meting nodig hebt; MCPTox-servers hebben die niet per definitie en focussen op tool poisoning. Voor een eerste validatie van detectie-correctheid heb je gecontroleerde grond nodig; MCPTox is de juiste tweede stap voor externe validiteit.

D. Reflectief / afsluitend

11. “Een Fintech-bedrijf draait je tool, de scan is groen. Waarvoor waarschuw je?”

Lijn: Groen $\neq$ veilig. De scanner dekt een vooraf gedefinieerde set klassen, detecteert patronen niet intentie, en mist subtiele, contextuele en businesslogica-kwetsbaarheden. Het is een eerste filter in CI/CD, geen vervanging van menselijke review of pentest.

12. “Wat zou je anders doen met drie maanden extra?” *Lijn:* Tweede/derde reële case (MCPTox), meerdere reviewers voor een eerlijke handmatige baseline, stdio-transport, en semantische detectie naast regex. Eén of twee concrete dingen, geen waslijst.

Deel 3. OWASP MCP Top 10 — uit het hoofd

Je goedkoopste puntenwinst. De traceability-tabel in je ingediende tekst gebruikt nog de LLM-codes (LLM01, LLM07, LLM09); de correcte MCP-codes passen *beter* op je regels. Memoriseer deze mapping (staat ook klaar als back-up-slide):

- Tool poisoning (V5 / FUZZ-004) → **MCP03**
- Ontbrekende autorisatie (SAST-003 / FUZZ-003) → **MCP07**
- Privilege escalation → **MCP02**

- Prompt injection (SAST-001) → **Intent Flow Subversion**

Deel 4. De 47-minuten-val

Je stand van zaken citeert dat een handmatige MCP-review 8 tot 16 uur kost (Uproot2025), terwijl jouw eigen manuele procedure 47 minuten duurde. Kader dit zelf vóór ze het vragen: jouw 47 min is op een kleine, opzettelijk kwetsbare server met een handvol tools; de 8 tot 16 uur uit de literatuur geldt voor volwaardige productieservers. Dat verschil ondersteunt net je punt.

Strategie: bepaal zelf het narratief vóór de jury het doet. Leg je beperkingen (één case, circulaire benchmark, n=1 reviewer) proactief op tafel via je reflectieslide. Dan wordt het een gesprek over volwassen onderzoek in plaats van een kruisverhoor. Dat is het verschil tussen een 12 en een 15 op dit onderdeel.